Locating Current working directory of our project on the pc

In [30]:
```
import os
os.getcwd()
```

Out[30]:
```
'C:\\Windows\\system32\\1033\\MALARIA EDA'
```

In [2]:
```
#Getting interaction between the Operating System and pproject file directory on th
os.chdir('C:\\Windows\\System32\\1033\\MALARIA EDA')
```

In [3]:
```
import numpy as np
import pandas as pd
from pandas import Series, DataFrame
```

In [4]:
```
# Returns a list of pathnames(Files) matching provided string
import glob
glob.glob("C:\\Windows\\System32\\1033\\MALARIA EDA/Malaria data/*.csv")
```

Out[4]:
```
['C:\\Windows\\System32\\1033\\MALARIA EDA/Malaria data\\Malaria data 2020.csv',
 'C:\\Windows\\System32\\1033\\MALARIA EDA/Malaria data\\Malaria data 2021.csv',
 'C:\\Windows\\System32\\1033\\MALARIA EDA/Malaria data\\Malaria data 2022.csv',
 'C:\\Windows\\System32\\1033\\MALARIA EDA/Malaria data\\Malaria data 2023.csv',
 'C:\\Windows\\System32\\1033\\MALARIA EDA/Malaria data\\Malaria data 2024.csv']
```

Merging All the Data Frames(Master_df) and Adding a 'Year' column

In [5]:
```
import glob
import pandas as pd
import os  #  os for the path functions

def clean_columns(df):
    df.columns = df.columns.str.strip()  # remove leading/trailing spaces
    df.columns = df.columns.str.replace(r"\s+", " ", regex=True)  # normalize multi
    return df

# Collect all CSVs
all_files = glob.glob("C:\\Windows\\System32\\1033\\MALARIA EDA/Malaria data/*.csv"

dfs = []
for file in all_files:
    df = pd.read_csv(file)
    df = clean_columns(df)
   #, encoding="cp1252"
    # Extract year from file name (this needs to be inside the loop)
    year = os.path.splitext(os.path.basename(file))[0].split()[-1]
    df["Year"] = int(year)

    dfs.append(df)  # This also needs to be inside the loop

# Combine into one master DataFrame
master_df = pd.concat(dfs, ignore_index=True)
```

```
 51  TOTAL CASES FEMALE                  805 non-null    int64
 52  TOTAL CASES MALE                    805 non-null    int64
 53  GRAND TOTAL CASES                   805 non-null    int64
 54  <5YRS FEMALES TOTAL                 805 non-null    int64
 55  <5YRS MALE TOTAL                    805 non-null    int64
 56  < 5YRS GRAND TOTAL                  805 non-null    int64
 57  POPULATION IN 2014                  795 non-null    float64
 58  POPULATION IN 2024                  795 non-null    float64
 59  AREA(Km2)                           795 non-null    float64
 60  TOTAL POPULATION                    805 non-null    object
 61  POPULATION DENSITY(People per km2)  805 non-null    object
 62  CORDINATES (LATITUDE ° N)           735 non-null    float64
 63  CORDINATES (LONGITUDE ° E)          735 non-null    object
 64  Year                                805 non-null    int64
 65  10-19 YRS TOTAL                     161 non-null    float64
 66  29DAYS-4YRS MALE                    161 non-null    float64
 67  20+ (B/S &RDT) MALE                 161 non-null    float64
dtypes: float64(10), int64(53), object(5)
memory usage: 427.8+ KB
```

In [8]:
```
master_df.shape
```

Out[8]:
```
(805, 73)
```

In [29]:
```
master_df.groupby("Year").size()
```

Out[29]:
```
Year
2020    161
2021    161
2022    161
2023    161
2024    161
dtype: int64
```

Removing Unwanted Columns

In [7]:
```
# Dropping some columns
#Dropping some Columns
df.drop(columns = ["POPULATION IN 2014", "POPULATION IN 2024"
                           ], inplace = True)
```

In [8]:
```
master_df.drop( columns = "ANNUAL GROWTH RATE[Pt?=P0?×(1+r)t]", inplace = True)
```

In [9]:
```
master_df.drop( columns = "ANNUAL GROWTH RATE[Pt=P0×(1+r)t]", inplace = True)
```

In [61]:
```
master_df.info()
```

```
 51  TOTAL CASES FEMALE                  805 non-null    int64
 52  TOTAL CASES MALE                    805 non-null    int64
 53  GRAND TOTAL CASES                   805 non-null    int64
 54  <5YRS FEMALES TOTAL                 805 non-null    int64
 55  <5YRS MALE TOTAL                    805 non-null    int64
 56  < 5YRS GRAND TOTAL                  805 non-null    int64
 57  POPULATION IN 2014                  795 non-null    float64
 58  POPULATION IN 2024                  795 non-null    float64
 59  AREA(Km2)                           795 non-null    float64
 60  TOTAL POPULATION                    805 non-null    object
 61  POPULATION DENSITY(People per km2)  805 non-null    object
 62  CORDINATES (LATITUDE ° N)           735 non-null    float64
 63  CORDINATES (LONGITUDE ° E)          735 non-null    object
 64  Year                                805 non-null    int64
 65  10-19 YRS TOTAL                     161 non-null    float64
 66  29DAYS-4YRS MALE                    161 non-null    float64
 67  20+ (B/S &RDT) MALE                 161 non-null    float64
 68  ANNUAL GROWTH RATE[Pt=P0×(1+r)t]  161 non-null    object
dtypes: float64(11), int64(52), object(6)
memory usage: 434.1+ KB
```

Merging misfit columns (columns that failed to merge due to spacing)

In [11]:
```
# Dictionary of columns that should be merged
merge_map = {
    "20+ (B/S &RDT) MALE": "20+ (B/S & RDT) MALE",
    "29DAYS-4YRS MALE": "29DAYS - 4YRS MALE",
    "10-19 YRS TOTAL": "10-19YRS TOTAL",
}

for bad_col, good_col in merge_map.items():
    if bad_col in master_df.columns and good_col in master_df.columns:
        # Merge values where good_col is missing
        master_df[good_col] = master_df[good_col].fillna(master_df[bad_col])
        # Drop the redundant col
        master_df = master_df.drop(columns=[bad_col])
```

Checking Data types

In [14]:
```
master_df.info()
```